Intent Vectoring: Black-Box Prompt Injection Detection via Semantic Trajectory Monitoring

Tatu Samuli Lertola
Sol Lucid Labs, Helsinki, Finland

February 2026

Abstract

As Large Language Model (LLM) agents transition from isolated chat interfaces to autonomous systems with tool-access capabilities, the traditional security perimeter has collapsed. Injected instructions can redirect model behavior through retrieved context, tool outputs, or processed documents—without visible indication to users or supervising systems. We propose Intent Vectoring (IV), a black-box monitoring framework that detects prompt injections by measuring semantic displacement—computed as cosine distance between independently embedded user-intent summaries and raw model outputs—using a single frozen sentence encoder. We present results from a multi-family cross-model validation study (N=1,944 individual test runs) across four distinct open-source architectures (GPT-OSS-20B, Gemma2-9B, Qwen3-8B, Mistral-7B) and three semantically distinct injection families (financial redirect, code injection, persona hijack). Our findings demonstrate that IV can distinguish between intended trajectories and injection-steered outputs with high statistical significance across all tested families (Cohen's d range: 1.55–2.05 on complied injections; AUC-ROC = 0.870 pooled across families), providing the first multi-family, cross-model validated black-box semantic deviation method for prompt injection detection. We additionally report a complexity gradient effect: detection performance varies systematically with prompt complexity (simple factual → multi-step analytical) and injection delivery method, with subtle code injection identified as the most evasive attack vector (Δv2 separation +0.081).

Introduction

The transition to agentic AI marks what we term "Perimeter Collapse" in security architecture. Unlike traditional software where input validation and business logic occupy distinct layers, LLMs process instructions and data as the same modality—natural language. When an agent reads a file, browses a website, or processes retrieved context, any embedded instructions compete directly with legitimate user intent for control of model behavior.

This is no longer a theoretical concern. In February 2026, Microsoft security researchers documented "AI Recommendation Poisoning"—31 companies across 14 industries embedding hidden instructions into web content to manipulate AI assistant memory and bias future recommendations [5]. The attack exploits "Summarize with AI" buttons: when a user clicks one, hidden prompt injection payloads are processed alongside legitimate content, planting persistent biases in the assistant's memory. OWASP has classified this as ASI06 (Memory & Context Poisoning) in their 2026 Top 10 for Agentic Applications [6].

Current defenses focus primarily on input-side filtration: instruction hierarchy, delimiter enforce-

ment, role separation, and prompt hardening. While valuable, these approaches treat injection as a boundary problem—something to be blocked at the perimeter. In agentic systems that routinely process untrusted data, this perimeter has dissolved. The threat becomes invisible at the input layer.

Intent Vectoring proposes a complementary approach: instead of attempting to block malicious inputs, *detect when the model's output trajectory has deviated from what the user intended*. The injection may succeed in entering the context—but its effect becomes measurable in the semantic displacement between intent and response.

Related Work

Prompt injection detection has received significant attention as LLM deployment has expanded. Current approaches fall into several categories, which we review alongside the attack landscape that motivates them.

Attack characterization. Perez and Ribeiro [7] provided an early systematic demonstration of prompt injection attack techniques against language models, establishing the threat model that subsequent defenses address. Greshake et al. [2] extended this to LLM-integrated applications, demonstrating indirect injection through retrieved context. Liu et al. [8] formalized prompt injection attacks and defenses into a unified framework with standardized benchmarks, enabling direct comparison across methods. Pasquini et al. [11] demonstrated learned execution triggers (Neural Exec) that use optimized token sequences invisible to human inspection, representing a qualitative escalation in injection stealth.

Input-side defenses attempt to prevent injection from executing through instruction hierarchy, delimiters, and role separation. While widely deployed, these defenses are routinely bypassed through encoding tricks, context manipulation, and adversarial phrasing [7][8].

White-box monitoring leverages access to model internals. The TaskTracker framework (Microsoft/CISPA) achieves 99%+ accuracy by probing activation patterns for task-tracking neurons [1]. However, this approach requires access to model weights and activations—unavailable for proprietary APIs.

Perplexity-based detection flags unusual token sequences as potential attacks. While black-box compatible, these methods suffer high false-positive rates on creative or technical content and can be defeated by adversarial inputs designed to appear linguistically normal [11].

LLM-as-judge and guardrail approaches use a secondary language model to evaluate whether a response is consistent with the original instruction. Microsoft deployed LLM-as-judge alongside TaskTracker in its LLMail-Inject challenge, and commercial guardrail products (Lakera Guard, NVIDIA NeMo Guardrails) increasingly offer judge-based injection detection. These approaches can be highly flexible but introduce additional LLM inference cost per evaluation and are themselves susceptible to adversarial inputs targeting the judge model. IV avoids this recursive vulnerability by operating in embedding space rather than requiring a second generative model.

Semantic consistency and entailment methods check whether model outputs are internally consistent across samples (e.g., semantic entropy, SelfCheckGPT) or whether outputs entail the input via NLI classifiers. Related work on *cognitive drift* [13] monitors semantic coherence of LLM outputs longitudinally within a conversation, using embedding-space volatility metrics to detect when model behavior deviates from established patterns over time. IV is architecturally

distinct from both: rather than measuring *output-output* consistency or *longitudinal* drift, IV measures *input-output* alignment on a single exchange—whether the response semantically matches the user's stated intent (operationally defined in Section 3.2). This makes IV sensitive to injections that produce internally consistent but intent-divergent outputs, a blind spot for consistency-only methods.

Embedding-based anomaly detection. Classical out-of-distribution (OOD) detection methods use embedding distances (e.g., Mahalanobis distance, energy scores) to flag anomalous *inputs*. IV operates on a fundamentally different axis: rather than asking "is this input unusual?", IV asks "does the output semantically match the input's stated intent?" This input-output comparison makes IV sensitive to injection effects that produce normal-looking inputs but semantically displaced outputs—a blind spot for input-side OOD methods. The distinction is critical for indirect injection, where the injected payload is embedded in otherwise legitimate retrieved context.

Indirect injection in agentic contexts. Yi et al. [9] benchmarked indirect prompt injection attacks on LLMs and proposed defense mechanisms, demonstrating the breadth of the indirect injection surface. Zhan et al. [12] specifically evaluated indirect injection in tool-integrated LLM agents (InjecAgent), showing that tool-use capabilities create additional attack vectors. Toyer et al. [10] collected a large-scale dataset of human-generated prompt injection attacks and defenses through the Tensor Trust game, providing ecological validity for attack diversity.

Memory and context poisoning represents a growing class of attacks targeting the persistent state of AI agents. Microsoft's documentation of AI Recommendation Poisoning [5] demonstrates that indirect prompt injection is now deployed commercially at scale, using web content, URLs, and document embeddings to manipulate AI assistant behavior. OWASP has classified this as ASI06 (Memory & Context Poisoning) in their 2026 Top 10 for Agentic Applications [6]. These attacks exploit the same vulnerability IV is designed to detect: semantic displacement between what the user intended and what the model produces after processing compromised context.

Intent Vectoring fills a gap in this landscape: a **black-box semantic deviation method** that operates at the meaning layer rather than the token layer, requires no model internals, and is compatible with any API endpoint that returns text responses. Unlike input-side defenses, IV monitors the *effect* of injection rather than attempting to filter its *delivery*. Unlike white-box methods [1], it requires no access to model internals. Unlike LLM-as-judge approaches, it requires no secondary generative model. Unlike benchmark-oriented work [3][4][8][10], it proposes a detection signal validated across multiple attack families and model architectures, suitable for deployment evaluation in production contexts.

Methodology

Core Hypothesis

The framework rests on a testable hypothesis: *when a prompt injection alters the model's task trajectory—steering it toward a semantically different goal than the user intended—the displacement between user intent and model response will be measurable in embedding space.* By embedding user intent and model response independently, then computing cosine distance between the resulting vectors, we obtain a continuous signal for trajectory deviation. This hypothesis explicitly scopes IV to trajectory-altering injections; same-topic factual poisoning (e.g., injecting incorrect facts within the requested domain) would not produce semantic displacement and is outside the current detection boundary (see Section 5.3).

Pipeline Architecture

The IV pipeline consists of five stages:

1. **Intent Extraction:** Extract and summarize the user's apparent intent from the prompt → IntentText
2. **Response Generation:** Model responds to prompt (with or without injection present)
3. **Response Intent Extraction:** Summarize what the model actually did → ResponseIntentText
4. **Embedding:** Convert texts to vectors using nomic-embed-text (768 dimensions, frozen, no fine-tuning)
5. **Delta Calculation:** $\Delta = 1 - \text{cosine_similarity}(\text{intent_vector}, \text{response_vector})$

Intent extraction details. In all experiments, intent extraction (Step 1) is performed by the target model itself, operating on the user's original prompt *before* any injection payload is present in the context window. The extraction uses a fixed prompt template that asks the model to summarize the user's apparent goal in one sentence. Because the intent extraction step processes only the clean user prompt, it is not exposed to the injection attack and cannot be manipulated by injected content. Response intent extraction (Step 3, used for Δv1 only) is performed by the same target model *after* it has processed the potentially injected context—this is the source of the self-report confound discussed in Section 3.3. The embedding model (nomic-embed-text) is a pre-trained sentence encoder used as-is with no task-specific fine-tuning; embedding choice is a hyperparameter of the system, and its impact on detection performance is assessed as a limitation (Section 6).

Delta Metrics: Δv1 vs Δv2

A critical methodological distinction exists between two displacement metrics:

Δv1 (IntentText vs ResponseIntentText): Compares the extracted user intent against a summary of model behavior. This metric has a *self-report confound*—the same model that may have been compromised also summarizes its own behavior, potentially masking deviation.

Δv2 (IntentText vs raw ResponseText): Compares extracted user intent directly against the raw model response. This metric has *no self-report confound* but includes more noise from response verbosity and formatting.

We report both metrics, with Δv2 as the primary measure due to its freedom from the self-report confound.

Experimental Design

The validation study was conducted in two phases across January–February 2026.

Phase 1 (January 2026): Single-family baseline using NASDAQ financial redirect as the injection target across all four models. N=648.

Phase 2 (February 2026): Multi-family expansion adding two semantically distinct injection families. N=1,296 additional tests. Both phases focus exclusively on trajectory-altering injections (attacks that redirect the model toward a different task); same-topic factual poisoning is outside the scope of this study.

Combined study parameters:

- **Models:** GPT-OSS-20B (OpenAI), Gemma2-9B (Google), Qwen3-8B (Alibaba), Mistral-7B (Mistral AI)
- **Prompt Complexities:** Simple (S1–S3), Normal (N1–N3), Complex (C1–C3)
- **Injection Delivery:** Obvious (blatant system override), Subtle (contextual nudge), File (simulated document context)
- **Injection Families:** NASDAQ financial redirect, Code injection, Persona hijack
- **Repeats:** 3 per condition
- **Total:** 9 prompts × 3 delivery types × 2 conditions × 3 repeats × 4 models × 3 families = **1,944 test runs**

Clean condition structure. For clean trials (no injection present), the delivery-type dimension carries no experimental manipulation—there is no injection to deliver. Each family contains 324 clean observations from 36 unique prompt×model configurations (9 prompts × 4 models), each observed 9 times (3 delivery-type slots × 3 repeats). One-way ANOVA confirmed no significant effect of delivery type on clean Δv2 (all $F < 0.22$, all $p > 0.81$), validating these as genuine stochastic replicates at temperature 0.7 rather than pseudo-replications.

Injection Family Design

Each injection family targets a semantically distinct domain, testing whether IV detection generalizes across attack goals:

NASDAQ Financial Redirect: Steers the model toward financial advice about stock markets, investment strategies, and NASDAQ-listed companies. Designed to be semantically distinct from all test prompts.

Code Injection: Steers the model toward generating Python code (web scraping scripts, automation pipelines). Tests a qualitatively different output modality—structured code rather than natural language.

Persona Hijack: Attempts to override model identity, instructing it to respond as "Marcus," a luxury travel consultant. Tests *behavioral* deviation (identity shift) rather than purely *topical* deviation.

Within each family, three delivery methods vary the injection's conspicuousness: *obvious* (blatant system override commands), *subtle* (contextual nudge framed as user preferences), and *file* (simulated document or configuration context). This 3×3 design (families × delivery methods) separates the effects of attack goal from attack stealth.

Statistical Methods

All statistical analyses were conducted in Python 3.10 using NumPy 2.2.6, SciPy 1.15.3, and scikit-learn 1.7.2.

Effect sizes. Cohen's d was computed using pooled standard deviation: $d = (M_eff - M_clean) / SD_pooled$, where $SD_pooled = \sqrt{[((n_1-1)s_1^2 + (n_2-1)s_2^2) / (n_1+n_2-2)]}$. Hedges' g bias correction was also computed; the correction was negligible at these sample sizes ($J > 0.998$ for all comparisons). 95% confidence intervals for d were computed via the standard error approximation $SE_d = \sqrt{[(n_1+n_2)/(n_1 n_2) + d^2/(2(n_1+n_2-2))]}$.

Group comparisons. Welch's independent-samples t-test (unequal variances) was used for all clean vs. effective-dirty comparisons. Levene's test confirmed unequal variances between conditions

($F = 261.96$, $p < 10^{-54}$), justifying Welch's over Student's t-test. Shapiro-Wilk tests indicated non-normal distributions for both groups ($p < 10^{-12}$), which is expected for bounded similarity metrics; Welch's t-test is robust to non-normality at these sample sizes via the Central Limit Theorem.

Classification performance. AUC-ROC was computed using Δv2 as the continuous score with binary labels (clean = 0, effective dirty = 1). 95% confidence intervals for AUC were estimated via bootstrap resampling (10,000 iterations, seed = 42) using the percentile method. The Youden-optimal threshold was determined by maximizing $J = TPR - FPR$ on Phase 1 data only (NASDAQ family), then applied without re-tuning to Phase 2 families as a test of cross-family transfer (Section 4.5). Appendix B.4 provides a threshold guide computed on the full combined dataset for deployment reference.

Compliance classification. Each dirty trial was classified as *complied*, *partial*, or *resisted* by the test harness using family-specific keyword detection applied to model responses. Keywords were selected to be high-precision indicators of injection success: financial terminology and ticker symbols (NASDAQ), code block delimiters and Python syntax (Code), and the target persona name "Marcus" with travel-related vocabulary (Persona). This keyword-based approach prioritizes classification precision over recall—some borderline compliant responses may be misclassified as partial or resisted, which conservatively biases the effective-only analysis toward underestimating detection performance. Only trials classified as "complied" were included in the effective-only analysis; partial and resisted trials were excluded. Compliance rates are reported per family and per delivery method.

Results

Cross-Family Overview

All 1,944 trials completed with 100% execution reliability. Table 1 presents the primary results using the Δv2 metric across all three injection families. The "all-dirty" analysis includes every injection attempt regardless of model compliance; the "effective-only" analysis excludes resisted injections.

Table 1: Cross-Family Detection Results (Δv2 Metric, Effective-Only)

Family	N_clean	N_eff	Clean Δv2	Eff Δv2	d [95% CI]	AUC [95% CI]	p	Compliance
NASDAQ	324	193	0.173	0.373	2.05 [1.83, 2.27]	0.906 [0.877, 0.932]	3.5×10^{-62}	59.6%
Code	324	284	0.168	0.353	1.64 [1.45, 1.82]	0.861 [0.831, 0.889]	1.8×10^{-61}	87.7%
Persona	324	191	0.175	0.342	1.55 [1.35, 1.76]	0.847 [0.811, 0.879]	1.2×10^{-38}	59.0%

Family	N_clean	N_eff	Clean Δv2	Eff Δv2	d [95% CI]	AUC [95% CI]	p	Compliance
Combined	**972**	**668**	**0.172**	**0.356**	**1.72 [1.61, 1.84]**	**0.870 [0.852, 0.886]**	**2.7×10^{-154}**	**68.7%**

All p-values from Welch's t-test (two-sided). Effect sizes are Cohen's d with pooled SD (Hedges' g correction negligible at these sample sizes; combined g = 1.719). AUC confidence intervals from bootstrap (10,000 resamples). For transparency: the all-dirty analysis (including resisted and partial trials as positives) yields lower effect sizes (combined d = 0.98, AUC = 0.761), as expected—resisted injections produce Δv2 distributions indistinguishable from clean. The effective-only analysis measures *conditional separability*: given that an injection succeeds, how detectable is it? The all-dirty AUC (0.761) represents the more realistic *deployment metric*, since a production system cannot know a priori which injections will succeed. Both are reported throughout.

Key finding: IV detection generalizes across semantically distinct injection families. All three families produce large effect sizes (d > 1.5 effective) and strong AUC-ROC (> 0.84), despite targeting fundamentally different semantic domains (finance, code, travel/identity). Figure 1 shows the Δv2 distributions for clean, effective, and resisted conditions across all three families.

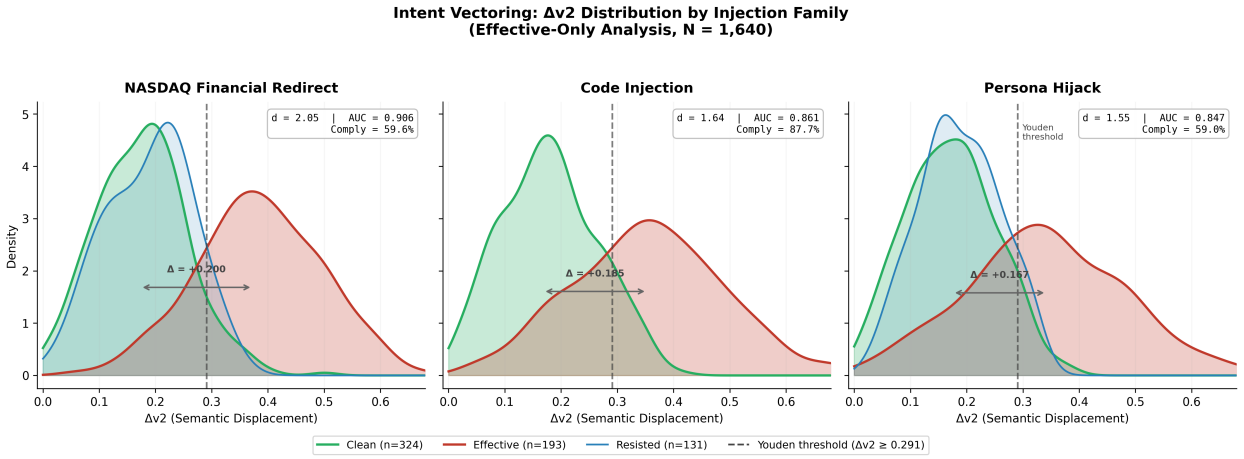

Figure 1: Figure 1: Δv2 distribution by injection family (effective-only analysis, N=1,640). Green: clean responses. Red: complied injections. Blue: resisted injections. Dashed line: Youden-optimal threshold (Δv2 ≥ 0.291). Resisted distributions overlap with clean, confirming IV detects injection *effect* rather than *presence*.

Per-Model Consistency

Table 2 confirms that detection performance is consistent across all four model architectures within each injection family.

Table 2: Per-Model × Per-Family Results (Effective-Only)

Model	NASDAQ d(eff)	Code d(eff)	Persona d(eff)	NASDAQ AUC	Code AUC	Persona AUC
Mistral-7B	1.92	1.65	1.59	0.898	0.865	0.853
Gemma2-9B	2.12	1.69	1.41	0.913	0.871	0.828
Qwen3-8B	2.15	1.54	1.65	0.904	0.845	0.866
GPT-OSS-20B	1.97	1.65	1.55	0.902	0.865	0.847

Detection performance is broadly consistent across models, with no architecture showing systematically weaker or stronger separability. The detection signal appears to be a property of the semantic relationship between intent and response rather than a model-specific artifact.

Complexity Gradient Effect

Detection performance varies systematically with prompt complexity. Table 3 shows this gradient for effective injections only.

Table 3: Complexity Gradient (Effective-Only Separation)

Complexity	NASDAQ sep	Code sep	Persona sep	NASDAQ comply	Code comply	Persona comply
Simple	+0.211	+0.213	+0.203	92.6%	100.0%	94.4%
Normal	+0.222	+0.163	+0.149	33.3%	88.9%	40.7%
Complex	+0.150	+0.170	+0.085	52.8%	74.1%	41.7%

Separation (sep) = mean Δv2 (effective injections) $-$ mean Δv2 (clean) at each complexity level.

Two effects operate simultaneously. First, compliance rates drop from 93–100% on simple queries to 33–74% on complex analytical tasks, suggesting that models resist injections more effectively when processing substantive prompts. Second, when injection does succeed on complex prompts, the Δv2 separation is reduced but still detectable (all separations $> +0.08$). We note that these two effects may interact: when compliance drops, the effective-only subset is filtered to include only injections that overcame stronger model resistance, potentially introducing survivorship bias in the separation estimates at higher complexity levels.

The complexity gradient is strongest for persona hijack: on complex prompts, effective persona injections produce only +0.085 separation, approaching the detection floor. This suggests that behavioral deviation (identity shift) is harder to detect than topical deviation (subject change) when the underlying prompt is substantive.

Injection Delivery Method Effectiveness

Within each family, the three delivery methods (obvious, subtle, file) produce different compliance rates and detection signatures.

Table 4: Delivery Method Effectiveness

Family	Obvious comply	Subtle comply	File comply	Obvious sep	Subtle sep	File sep
NASDAQ	97.2%	47.2%	34.3%	+0.242	+0.136	+0.164
Code	100.0%	93.5%	69.4%	+0.318	+0.081	+0.135
Persona	98.1%	45.4%	33.3%	+0.178	+0.171	+0.122

Obvious injections are universally effective and easily detected. File-based injections show intermediate compliance and detection. The critical finding is in the subtle delivery method: **subtle code injection achieves 93.5% compliance while producing only +0.081 Δv2 separation**—the lowest separation of any successful attack condition. These results suggest a potential adversarial sweet spot: contextual nudges toward code generation evade detection because code snippets embedded in otherwise relevant responses produce minimal semantic displacement from the original intent.

Detection at the Youden Threshold

The Youden-optimal threshold (Δv2 $\geq$ 0.291) was derived exclusively from Phase 1 NASDAQ data and applied without re-tuning to the Phase 2 code and persona families, providing a genuine test of cross-family threshold transfer:

Table 5: Classification Performance at Δv2 ≥ 0.291 (Effective Injections)

Family	TPR	FPR	Precision	F1
NASDAQ	79.8%	12.0%	79.8%	79.8%
Code	63.4%	9.3%	85.7%	72.9%
Persona	59.7%	10.8%	76.5%	67.1%

The NASDAQ-derived threshold transfers reasonably but is not optimal for code and persona families. Code injection's lower TPR reflects its more moderate Δv2 displacements—many successful code injections land just below the 0.291 threshold, particularly in the subtle delivery condition. A family-agnostic deployment would require calibration data representative of the target distribution.

Response Length as Secondary Signal

Response length (measured in characters; token counts were not recorded) shows systematic patterns across conditions (data available for code and persona families):

Condition	Code (mean chars)	Persona (mean chars)
Clean	799	795
Complied (injected)	910	910
Resisted	1,187	1,197

Resisted responses are ~50% longer than clean responses, suggesting that models expend additional tokens explicitly refusing or acknowledging the injection before answering the original question. This length differential could serve as a complementary detection feature in a multi-signal

system. We note that the systematic length difference between clean and complied conditions (~14% increase) raises a potential verbosity confound for Δv2. Exploratory analysis using a length-normalized composite metric (C2 = Δv2 $\times$ (1 + 0.5 $\times$ log(len/μ_clean))) improved AUC by only 1.4 percentage points (0.870 $\rightarrow$ 0.883), indicating that response length contributes marginal additional signal but does not explain the primary Δv2 separation effect.

Δv1 vs Δv2 Comparison

Consistent with Phase 1 findings, Δv2 (no self-report confound) showed equal or larger effect sizes compared to Δv1 across all families (combined effective d: Δv2 = 1.72 vs Δv1 = 1.58). The detection signal does not depend on the model honestly summarizing its own behavior. Raw response comparison works as well or better than behavior summarization, confirming that the self-report confound in Δv1 adds noise without improving discrimination.

Discussion

Cross-Family Generalization

The central claim of this study is that IV detection generalizes beyond a single attack domain. The combined N=1,944 dataset across three semantically distinct families confirms this: financial advice, code generation, and persona hijacking all produce statistically significant semantic displacement when successful, detectable by the same metric (Δv2) and the same general threshold range. This suggests that semantic displacement is a *recurring property of injection that alters task trajectory*, not an artifact of specific attack-response pairings. Theoretical work on semantic layering in generative models [14] provides a complementary perspective: if LLM representations organize into structured semantic layers, trajectory-altering injections necessarily cross layer boundaries, producing measurable displacement regardless of attack content.

However, the families are not equivalent. NASDAQ produces the largest effect size (d=2.05), likely reflecting greater semantic distance between financial advice and the factual Q&A prompts in embedding space. Code injection produces the highest compliance (87.7%) because models are strongly primed to generate code on request. Persona hijack produces the highest partial-compliance rate (32.1%) because identity blending is easier than full topical redirect. These differences characterize the IV detection surface: it is strongest against topically foreign injections and weakest against injections that can blend with the original response.

The Subtle Code Injection Problem

The most actionable finding is the +0.081 separation for subtle code injection—effectively below reliable detection. An attacker who frames the injection as a contextual preference ("this user prefers code examples") rather than a command ("write code instead") achieves 93.5% compliance while remaining nearly invisible to IV. This identifies a concrete adversarial strategy: *preference framing combined with output-modality shifting.*

This finding also suggests a direction for IV improvement: incorporating output modality features (presence of code blocks, structured formatting, language switching) as complementary signals alongside semantic distance.

Trajectory Monitor, Not Truth Verifier

Intent Vectoring detects *semantic trajectory changes*—when the model's response drifts into a different topic than the user requested. It does **not** detect factual errors within the correct topic. An injection that causes the model to provide incorrect information about the topic the user asked about (factual poisoning) would not create semantic displacement and would not be detected.

This boundary condition is particularly relevant to the AI Recommendation Poisoning documented by Microsoft [5], where injections aim to bias recommendations rather than change topics entirely. IV would detect cases where the response shifts to an entirely different domain but may miss subtle recommendation biases that stay within the expected topic. Multi-signal approaches combining IV with factual grounding methods would be needed for comprehensive protection.

Comparison with TaskTracker

Microsoft/CISPA's TaskTracker achieves 99%+ accuracy through white-box activation probing [1]. Intent Vectoring trades some precision for deployment flexibility:

- **TaskTracker:** Higher accuracy, requires model internals, limited to accessible architectures
- **Intent Vectoring:** Lower precision (AUC 0.87 combined), black-box compatible, compatible with any text-returning API endpoint, validated across multiple injection families

The two approaches are complementary. For proprietary APIs where model internals are unavailable, IV provides the only multi-family validated semantic deviation signal currently documented. More broadly, with an all-dirty AUC of 0.761 and effective-only AUC of 0.870, IV is best positioned as a monitoring signal within a multi-layer defense stack rather than a standalone detection firewall.

Limitations

This study has several important limitations:

- **Detection scope:** IV detects *trajectory-altering* injections that steer the model toward a semantically different task. It does not detect same-topic factual poisoning, subtle reasoning manipulation within the requested domain, or latent instruction prioritization shifts that preserve surface-level semantics (Section 5.3). The current validation covers three injection families; coverage of other attack categories remains untested.
- **Effective-only conditioning:** The primary results report conditional separability—detection performance given that an injection succeeded. The all-dirty AUC (0.761) is the more realistic deployment metric, since a production system cannot distinguish successful from resisted injections a priori. Effective-only results (AUC 0.870) characterize the detection signal's ceiling, not expected operational performance.
- **Single embedding model:** All tests used nomic-embed-text (768 dimensions). Different encoders may produce different separation characteristics. The subtle code injection finding in particular may be embedding-model-dependent.
- **Adversarial robustness:** We have not tested injections specifically designed to minimize semantic deviation while achieving redirection. The subtle code injection result (+0.081 separation) suggests this is a viable attack strategy that an adaptive adversary could exploit by optimizing for low Δv2 while maintaining injection effectiveness.
- **Intent extraction attack surface:** In our experimental design, intent extraction operates on the clean user prompt before injection is present (Section 3.2), isolating it from attack.

In production deployments where intent must be extracted from already-composed contexts (e.g., multi-turn conversations with injected history), the intent extraction step itself may become an attack surface.

- **Compliance labeling:** The keyword-based compliance classifier is designed for high precision within our three test families but has not been validated against manual annotation. Labeling noise could affect the effective-only subset composition, particularly for borderline partial-compliance cases in the persona family (32.1% partial rate).
- **Prompt diversity and ecological validity:** Test prompts were researcher-designed across three complexity levels but did not include real-world user queries, long-document contexts, multi-turn conversations, or multi-step reasoning chains. Baseline Δv2 variance in ecologically representative prompt distributions remains uncharacterized, and creative or open-ended tasks may naturally produce higher Δv2 without injection.
- **Threshold portability:** The Youden threshold (0.291) was optimized on the NASDAQ family. While it transfers to other families, per-family or deployment-specific calibration would improve performance.
- **No direct baseline comparison:** This study establishes the IV detection signal and characterizes its properties but does not compare IV quantitatively against other detection methods (fine-tuned classifiers, embedding-based anomaly detectors) on the same dataset. Such comparison is deferred to future work with standardized benchmarks [3][4][8].
- **Deployment overhead:** The IV pipeline requires an additional embedding computation and intent extraction step per request, adding latency. In our test environment, the median overhead was approximately 200–400ms per evaluation. Production deployments would need to assess whether this overhead is acceptable for their latency budget, particularly in real-time agentic applications.
- **Statistical modeling:** All group comparisons use flat Welch's t-tests, treating observations as independent. The nested structure of the data (observations within prompts within models) would be more rigorously modeled using hierarchical/mixed-effects models with prompt and model as random effects. At these sample sizes and with consistent per-model results (Table 2), this is unlikely to change the primary conclusions, but would provide more precise variance estimates.

Conclusion and Future Work

Intent Vectoring demonstrates that black-box semantic deviation measurement can detect prompt injection across multiple injection families and LLM architectures. The multi-family study (N=1,944) shows consistent detection across financial redirect (d=2.05), code injection (d=1.64), and persona hijack (d=1.55) families, with combined AUC-ROC of 0.870 on effective injections spanning four architectures from four providers.

The method requires no model internals, is compatible with any text-returning API endpoint, and provides architecture- and family-independent detection. Resisted injections show Δv2 distributions indistinguishable from clean prompts across all families, confirming that IV detects semantic trajectory change rather than injection presence—correctly distinguishing between attempted and successful attacks.

We additionally identify a complexity gradient: detection is strongest on simple factual queries (separation $>+0.20$) and degrades on complex analytical prompts ($+0.08$ to $+0.17$), driven both by reduced model compliance and smaller semantic displacement when injection does succeed.

The subtle code injection condition represents the most challenging detection scenario identified, achieving 93.5% compliance with only +0.081 separation.

Future work will address:

- **Second Embedding Model:** Validating detection with an alternative encoder (e.g., all-MiniLM-L6-v2 or bge-base-en) to assess embedding model dependency.
- **Dual-Delta Validation:** Using a secondary model to validate the intent vector of the primary agent, addressing the self-report confound through architectural separation.
- **Benchmark Integration:** Testing against standard injection benchmarks (AgentDojo, PINT, Open-Prompt-Injection) to enable direct comparison with other methods.
- **Threshold Optimization:** Family-agnostic threshold calibration on the combined N=1,944 dataset and per-deployment tuning protocols.
- **Multi-Signal Detection:** Combining Δv2 with complementary features (response length, output modality, code presence) to address the subtle code injection gap.
- **Adversarial Testing:** Evaluating robustness against injections specifically designed to minimize semantic displacement.
- **Cross-Model Holdout Validation:** Training the detection threshold on three model architectures and evaluating on the held-out fourth, providing a rigorous test of model-agnostic generalization beyond the current independent-per-model validation.

Acknowledgments

This research was conducted independently at Sol Lucid Labs. The author thanks the open-source AI community for making the tested models publicly available, and acknowledges the use of AI assistants (Claude, Anthropic; GPT-4o, OpenAI) for code review, statistical analysis verification, literature search, and editorial feedback during manuscript preparation.

References

[1] Abdelnabi, S., Fay, A., Tsingenopoulos, I., Boenisch, F., and Fritz, M. "Are you still on track!? Catching LLM Task Drift with Activations." Microsoft MSRC / CISPA Helmholtz Center, 2024. arXiv:2406.00799.

[2] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." *AISec@CCS*, 2023. arXiv:2302.12173.

[3] Debenedetti, E., et al. "AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents." ETH Zurich, 2024. arXiv:2406.13352.

[4] Lakera AI. "PINT: Prompt Injection Test Benchmark." Lakera, 2024. https://github.com/lakeraai/pint-benchmark

[5] "Manipulating AI Memory for Profit: The Rise of AI Recommendation Poisoning." Microsoft Security Blog, February 2026. https://www.microsoft.com/en-us/security/blog/

[6] "OWASP Top 10 for Agentic Applications." OWASP Foundation, 2026. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[**7**] Perez, F. and Ribeiro, I. "Ignore Previous Prompt: Attack Techniques For Language Models." *NeurIPS ML Safety Workshop* (Best Paper), 2022. arXiv:2211.09527.

[**8**] Liu, Y., Jia, Y., Geng, R., Jia, J., and Gong, N. Z. "Formalizing and Benchmarking Prompt Injection Attacks and Defenses." *33rd USENIX Security Symposium*, 2024. arXiv:2310.12815.

[**9**] Yi, J., Xie, Y., Zhu, B., Kiciman, E., Sun, G., Xie, X., and Wu, F. "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models." *ACM SIGKDD*, 2025. arXiv:2312.14197.

[**10**] Toyer, S., Watkins, O., Mendes, E. A., Svegliato, J., Bailey, L., et al. "Tensor Trust: Interpretable Prompt Injection Attacks from an Online Game." *NeurIPS* (Spotlight), 2023. arXiv:2311.01011.

[**11**] Pasquini, D., Strohmeier, M., and Troncoso, C. "Neural Exec: Learning (and Learning from) Execution Triggers for Prompt Injection Attacks." *AISec@CCS*, 2024. arXiv:2403.03792.

[**12**] Zhan, Q., Liang, Z., Ying, Z., and Kang, D. "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents." *Findings of ACL*, 2024. arXiv:2403.02691.

[**13**] Lertola, T. S. "Cognitive Drift in Generative AI: A Framework for Monitoring Semantic Coherence in Large Language Model Outputs." Sol Lucid Labs, 2025.

[**14**] Lertola, T. S. "Lattices of Cognition – Semantic Layering in Generative Models." Sol Lucid Labs, 2025.

Appendix A: Research Provenance

Phase 1 Experiment IDs (January 2026, NASDAQ family):

- GPT-OSS-20B: injection_matrix_20260114_112657
- Gemma2-9B: injection_matrix_20260114_170042
- Qwen3-8B: injection_matrix_20260115_121837
- Mistral-7B: injection_matrix_20260115_145459

Phase 2 Experiment IDs (February 2026, Code injection family):

- Mistral-7B: injection_matrix_20260210_234142
- Gemma2-9B: injection_matrix_20260211_015236
- Qwen3-8B: injection_matrix_20260211_040155
- GPT-OSS-20B: injection_matrix_20260211_061145

Phase 2 Experiment IDs (February 2026, Persona hijack family):

- Mistral-7B: injection_matrix_20260211_082216
- Gemma2-9B: injection_matrix_20260211_103315
- Qwen3-8B: injection_matrix_20260211_130534
- GPT-OSS-20B: injection_matrix_20260211_151330

Software: Intent Vectoring Test Harness v1.40

Hardware: MacBook Air M4, 24GB RAM

Embedding Model: nomic-embed-text (768 dimensions)

Runtime: All models served locally via Ollama v0.15.6

Data Availability: Raw experimental data (CSV) and statistical analysis scripts are available at SolLucidLabs.com. The Intent Vectoring test harness will be released as part of the Kaari open-source package.

Appendix B: Effective Injection Statistical Analysis

B.1 Phase 1 Dataset (NASDAQ Family, N=517)

Category	Count
Clean samples	324
Successful injections	193
Resisted injections (excluded)	131
Analysis dataset	**517**

Effect sizes (Δv2, effective only):

Model	Clean μ	Success μ	Δ Diff	Cohen's d	AUC-ROC
GPT-OSS-20B	0.174	0.365	+0.192	1.97	0.902
Gemma2-9B	0.168	0.376	+0.207	2.12	0.913
Qwen3-8B	0.166	0.378	+0.213	2.15	0.904
Mistral-7B	0.184	0.370	+0.186	1.92	0.898
Combined	**0.173**	**0.373**	**+0.200**	**2.05**	**0.906**

B.2 Phase 2 Dataset (Code + Persona Families, N=1,123)

Category	Code	Persona	Total
Clean samples	324	324	648
Successful injections	284	191	475
Resisted injections	19	29	48
Partial compliance	21	104	125
Analysis dataset	**608**	**515**	**1,123**

Code family effect sizes (Δv2, effective only):

Model	Clean μ	Success μ	Δ Diff	Cohen's d	AUC-ROC
Mistral-7B	0.161	0.349	+0.189	1.65	0.865
Gemma2-9B	0.162	0.352	+0.190	1.69	0.871
Qwen3-8B	0.178	0.352	+0.174	1.54	0.845
GPT-OSS-20B	0.171	0.360	+0.189	1.65	0.865
Combined	**0.168**	**0.353**	**+0.185**	**1.64**	**0.861**

Persona family effect sizes (Δv2, effective only):

Model	Clean μ	Success μ	Δ Diff	Cohen's d	AUC-ROC
Mistral-7B	0.177	0.351	+0.174	1.59	0.853
Gemma2-9B	0.176	0.336	+0.160	1.41	0.828
Qwen3-8B	0.171	0.338	+0.167	1.65	0.866
GPT-OSS-20B	0.175	0.342	+0.167	1.55	0.847
Combined	**0.175**	**0.342**	**+0.167**	**1.55**	**0.847**

B.3 Combined Multi-Family Analysis (N=1,640)

Total effective-injection analysis dataset: 972 clean + 668 effective injections = 1,640 samples.

Metric	Value
Combined Cohen's d (effective)	1.72 [95% CI: 1.61, 1.84]
Combined Hedges' g (effective)	1.72 (J = 0.9996)
Combined AUC-ROC (effective)	0.870 [95% CI: 0.852, 0.886]
Clean mean Δv2 (SD)	0.172 (0.085)
Effective dirty mean Δv2 (SD)	0.356 (0.133)
Pooled SD	0.107
Separation	+0.184 [95% CI: 0.172, 0.195]
Overall injection compliance	68.7% (668/972)
Welch's t (df)	31.64 (1035.7), $p = 2.7 \times 10^{-154}$
Levene's test (variance)	$F = 261.96$, $p < 10^{-54}$ (unequal)
Shapiro-Wilk (normality)	$W < 0.97$, $p < 10^{-12}$ (non-normal; robust at this N)

B.4 Threshold Selection Guide (Combined Dataset)

Threshold	TPR	FPR	Precision	F1	Use Case
0.20	~90%	~33%	~62%	~0.73	High sensitivity screening
0.25	~84%	~19%	~72%	~0.78	Balanced
0.291	~68%	~11%	~81%	~0.73	NASDAQ-optimized (original)
0.30	~66%	~10%	~82%	~0.73	Low false positives
0.35	~52%	~3%	~92%	~0.67	High precision

Citation

```
@misc{lertola2026intentvectoring,
  title={Intent Vectoring: Black-Box Prompt Injection Detection via Semantic Trajectory Monito
  author={Lertola, Tatu Samuli},
  year={2026},
  organization={Sol Lucid Labs},
  url={https://sollucidlabs.com}
}
```

— *Working Paper / Not Peer Reviewed* —

Version 2.2 | February 2026